

NetGuard-Suite

Project architecture & networking primer

This document summarises how the lab app is structured (layers, classes, methods, dashboard behaviour) and includes short background on Streamlit, TCP/UDP/ICMP, and SNMP for context.

1. What is Streamlit?

Streamlit is an open-source Python framework for building data applications and dashboards. You write a normal Python script: the framework runs it from top to bottom whenever the user interacts with the page (clicks a button, moves a slider, etc.). Each interaction triggers a rerun of the script so the UI stays in sync with your variables.

Why it is useful

- Very little boilerplate: no separate HTML/CSS/JS project for simple tools.
- Rich widgets (buttons, tables, charts) with a few lines of code.
- Ideal for teaching, prototypes, and internal monitoring-style demos.
- Integrates well with pandas, matplotlib, and other Python libraries.

Typical uses in this project

- `st.set_page_config` — tab title and page layout.
- `st.session_state` — keep the device inventory between reruns.
- `st.button` / `st.slider` / `st.selectbox` — user actions and parameters.
- `st.dataframe` — tabular node list and traffic rows.
- `st.pyplot` — pie chart of protocol mix.

Minimal examples (syntax patterns)

```
# Run the app from a terminal:
#   streamlit run app.py

import streamlit as st

st.title("My dashboard")
if st.button("Refresh"):
    st.session_state["count"] = st.session_state.get("count", 0) + 1
st.write("Count:", st.session_state.get("count", 0))
```

```
import pandas as pd

df = pd.DataFrame({"A": [1, 2], "B": [3, 4]})
st.dataframe(df, use_container_width=True)
```

2. TCP, UDP, and ICMP (definitions and differences)

TCP (Transmission Control Protocol)

Connection-oriented transport on top of IP. Two endpoints establish a session, data is delivered as a reliable byte stream: ordering is preserved, lost packets are retransmitted, and the receiver can throttle the sender. Uses port numbers to multiplex applications on

one host.

UDP (User Datagram Protocol)

Connectionless transport on top of IP. Sends self-contained datagrams with minimal overhead: no built-in retransmission or ordering guarantees. Good for latency-sensitive or one-shot messages where the application tolerates loss. Also uses ports.

ICMP (Internet Control Message Protocol)

A network-layer companion to IP (not a transport protocol like TCP/UDP). Used for control and diagnostics — e.g. echo request/reply ("ping"), destination unreachable, time exceeded. ICMP messages do not use TCP/UDP port numbers in the same way; they have types and codes instead.

Main differences at a glance

Aspect	TCP	UDP	ICMP
Layer	Transport (L4)	Transport (L4)	Network control (with IP)
Connections	Yes (reliable stream)	No (datagrams)	N/A (message types)
Reliability	Built-in	Application choice	Diagnostic / errors
Ordering	Preserved	Not guaranteed	N/A
Ports	Yes (src/dst ports)	Yes	No classic L4 ports
Typical use	Web, SSH, email	DNS, VoIP, gaming	Ping, traceroute, errors
Overhead	Higher	Lower	Low (short messages)

In NetGuard-Suite, flow rows are synthetic: TCP and UDP rows include a display Dst_Port; ICMP rows omit Dst_Port to mirror real captures where ICMP has no TCP/UDP port field.

3. SNMP for network management and monitoring

SNMP (Simple Network Management Protocol) is an application-layer protocol family, widely used to manage and monitor network equipment and servers in a standard way. It does not carry user traffic like web pages; it carries structured management data between a management station and agents on devices.

Core ideas

- Manager (NMS) — software that polls devices or receives notifications to build dashboards and alerts.
- Agent — process on a router, switch, server, or appliance that exposes management variables.
- MIB (Management Information Base) — logical catalogue of named objects (OIDs) the agent implements.
- OID (Object Identifier) — dotted number tree identifying one readable or writable value (e.g. interface octets, system uptime).

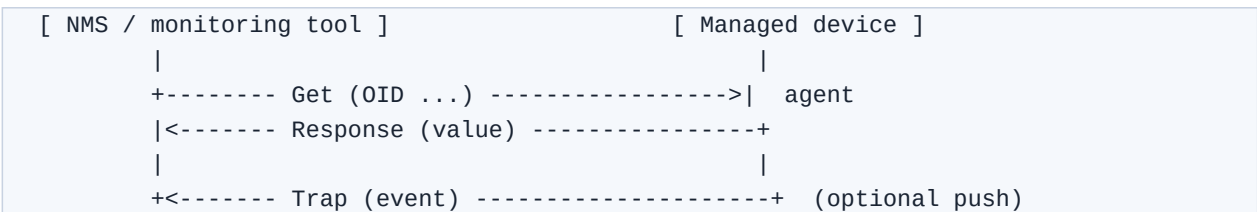
How SNMP supports operations teams

- Monitoring — periodic Get requests read counters and gauges (CPU load, memory, link utilisation, uptime) for graphs and SLAs.
- Fault detection — threshold crossings, link down, high error rates; SNMP traps/informs can push events to the manager without constant polling.

- Inventory and configuration — some MIBs expose model, serial, or software version; write-capable agents allow controlled Set (used carefully, often with extra auth).
- Capacity planning — long-term collection of interface and device metrics to plan upgrades.

Common message patterns (simplified)

- Get / GetNext / GetBulk — manager reads one or many OIDs from an agent (typical polling).
- Set — manager writes a value where the MIB and security policy allow it (configuration / control).
- Trap / Inform — agent asynchronously notifies the manager of an event (e.g. coldStart, linkDown).



Security note (real deployments)

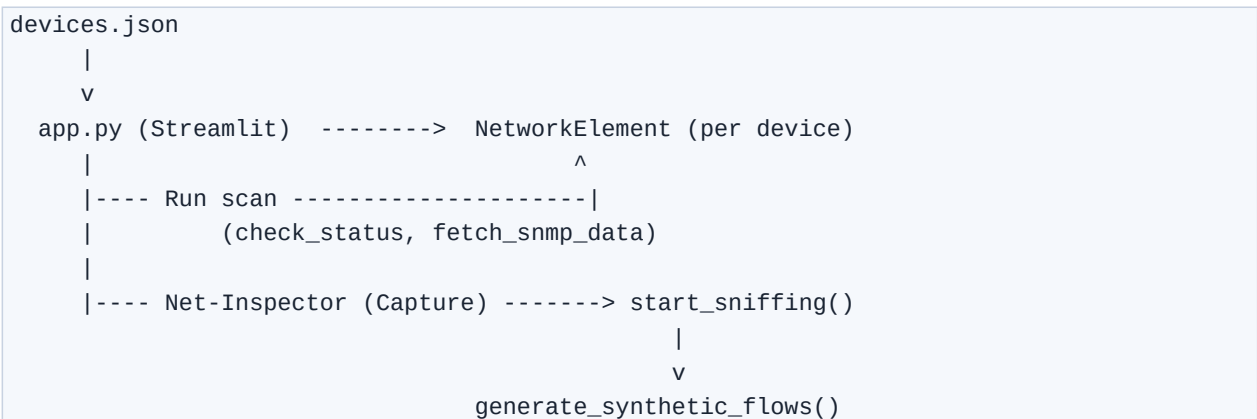
SNMPv1/v2c often use a simple community string (shared secret in cleartext). SNMPv3 adds user-based security (authentication and privacy). Production networks prefer v3 or restrict access with ACLs and dedicated management VLANs.

Relation to NetGuard-Suite

This lab does not open SNMP sockets or talk to real agents. The Run network scan action calls `fetch_snmp_data()`, which only simulates SNMP-style fields (uptime string, CPU percentage) using hints from `devices.json` plus random jitter — enough to teach the role of polling-style metrics in a NOC dashboard without the complexity of a full SNMP stack.

4. System overview

Three layers: (A) Streamlit script `app.py` (no domain class); (B) `NetworkElement` in `core/network_element.py`; (C) `generate_synthetic_flows` in `sim/traffic_generator.py`. Inventory is read from `devices.json`; the list of nodes lives in `st.session_state`.



5. Dashboard module: `app.py`

Purpose: educational NOC-style view — titles, captions, load inventory, simulated scan on all nodes, summary table (Node map), Net-Inspector (node pick, capture duration, synthetic flow table, protocol pie chart). No domain classes: orchestrates NetworkElement, pandas, matplotlib.

Main script flow (each Streamlit rerun)

- set_page_config — browser tab title and wide layout.
- Path constants — app directory and absolute path to devices.json.
 - _INVENTORY_SESSION_VERSION — bump to invalidate cached inventory in session_state.
- First load — parse JSON ({"devices": [...]} or root list), build NetworkElement list; on error, st.error and empty list.
- Run network scan — for each element: check_status() then fetch_snmp_data().
- Node map — build rows (hostname, IP, status, latency, uptime, CPU); st.dataframe.
 - Net-Inspector — selectbox, slider 2-10 s, capture button (disabled if offline); start_sniffing with all IPs; show table / errors.
- Pie chart — matplotlib on Protocol value_counts from traffic_log.
- Empty inventory — st.warning to add devices.json next to app.py.

6. Class NetworkElement (core/network_element.py)

Logical node: simulated reachability, SNMP-style fields, and traffic_log for Net-Inspector. Uses sim.traffic_generator.generate_synthetic_flows.

Module constants

```
_OUTAGE_PROBABILITY = 0.14
_OUTAGE_MIN_SECONDS = 1.5
_OUTAGE_MAX_SECONDS = 3.5
```

Tune how often and how long simulated outages last when check_status runs (while not already inside an outage window).

Instance attributes

```
ip_address, hostname    # shown in UI
status                  # 'Unknown' | 'Online' | 'Offline'
latency                 # float ms (0 if offline)
snmp_port               # from JSON; drives synthetic TCP/UDP Dst_Port
_uptime_hint, _cpu_base # internal hints for simulated metrics
uptime, cpu_load         # strings after scan ('N/A' if offline)
traffic_log              # list of flow dicts after capture
sniff_error              # str | None
_offline_until           # time.time() when outage ends
```

Methods

__init__(ip_address, hostname, snmp_port=16100, uptime_base=None, cpu_base=None)
— initialise state, hints, empty traffic_log, sniff_error None, _offline_until 0.

check_status() -> str — if inside outage window: Offline, latency 0; else maybe start new outage with _OUTAGE_PROBABILITY; else Online with random latency in [0.9, 24] ms.

seconds_until_online() -> Optional[float] — remaining outage seconds for UI '~Xs', else

None.

fetch_snmp_data() -> None — if not Online, uptime/cpu_load 'N/A'; else CPU from _cpu_base + jitter, uptime from hint.

start_sniffing(duration=5, inventory_ips=None) -> None — clear log; if Offline set error; else call generate_synthetic_flows and store rows or sniff_error.

7. Module sim/traffic_generator.py

No classes: synthetic UDP/TCP/ICMP-style rows without sockets or threads.

Constants

```
_MOCK_UDP_PORT_OFFSET = 10000
 MOCK_TCP_PORT_OFFSET = 15000
_PROTOCOL_WEIGHTS = (0.38, 0.42, 0.20) # UDP, TCP, ICMP
```

generate_synthetic_flows(...)

Returns (rows, None) or ([], error string). Validates inventory contains node_ip and a peer. n_flows = max(14, duration * uniform(7, 16)). For each row: weighted protocol, random peer, random direction. ICMP: no Dst_Port, Length 84. UDP/TCP: Length from headers + payload, Dst_Port set. Rows shuffled.

main() — CLI demo; run with: python -m sim.traffic_generator.

8. Packages core and sim

core/__init__.py exports NetworkElement in __all__. sim/__init__.py is a short package docstring only.

9. Data flow summary

devices.json -> NetworkElement list in session_state. Scan updates each node. Capture calls start_sniffing -> generate_synthetic_flows with all inventory IPs; traffic_log feeds the table and matplotlib pie chart for the selected node.